



Python Programming Fundamentals

Recursion

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. What is Recursion?
2. The Factorial Example
3. Tracing the Call Stack
4. Fibonacci Sequence
5. Sum of a List Recursively
6. When to Use Recursion
7. Recursion vs Iteration Comparison



Outline

- 1. What is Recursion?**
2. The Factorial Example
3. Tracing the Call Stack
4. Fibonacci Sequence
5. Sum of a List Recursively
6. When to Use Recursion
7. Recursion vs Iteration Comparison



1. What is Recursion?

Recursion is when a function calls **itself** to solve a smaller version of the same problem.



1. What is Recursion?

Recursion is when a function calls **itself** to solve a smaller version of the same problem.

Think of it like Russian dolls — each doll contains a smaller version of itself, until you reach the smallest one.



1. What is Recursion?

Recursion is when a function calls **itself** to solve a smaller version of the same problem.

Think of it like Russian dolls — each doll contains a smaller version of itself, until you reach the smallest one.

Every recursive function must have:

1. A **base case** — the simplest case that can be solved directly (no recursion)
2. A **recursive case** — breaks the problem down and calls itself with a smaller input



1. What is Recursion?

Recursion is when a function calls **itself** to solve a smaller version of the same problem.

Think of it like Russian dolls — each doll contains a smaller version of itself, until you reach the smallest one.

Every recursive function must have:

1. A **base case** — the simplest case that can be solved directly (no recursion)
2. A **recursive case** — breaks the problem down and calls itself with a smaller input

Without a base case, a recursive function calls itself forever → `RecursionError`:
maximum recursion depth exceeded



Outline

1. What is Recursion?
- 2. The Factorial Example**
3. Tracing the Call Stack
4. Fibonacci Sequence
5. Sum of a List Recursively
6. When to Use Recursion
7. Recursion vs Iteration Comparison



2. The Factorial Example

Factorial: $n! = n \times (n-1) \times (n-2) \times \dots \times 1$

- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
- $1! = 1$
- $0! = 1$ (by definition)



2. The Factorial Example



2. The Factorial Example

Factorial: $n! = n \times (n-1) \times (n-2) \times \dots \times 1$

- $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
- $1! = 1$
- $0! = 1$ (by definition)

Iterative version:

```
def factorial_iterative(n):  
    result = 1  
    for i in range(2, n + 1):  
        result *= i  
    return result
```

Recursive version:



2. The Factorial Example

```
def factorial(n):  
    if n <= 1:                # base case  
        return 1  
    return n * factorial(n - 1)  # recursive case  
  
print(factorial(5))    # 120  
print(factorial(0))    # 1
```



Outline

1. What is Recursion?
2. The Factorial Example
- 3. Tracing the Call Stack**
4. Fibonacci Sequence
5. Sum of a List Recursively
6. When to Use Recursion
7. Recursion vs Iteration Comparison



3. Tracing the Call Stack

factorial(4) step by step:

```
factorial(4)
  → 4 * factorial(3)
    → 3 * factorial(2)
      → 2 * factorial(1)
        → 1 (base case!)
          ← 2 * 1 = 2
            ← 3 * 2 = 6
              ← 4 * 6 = 24
                = 24
```



3. Tracing the Call Stack



3. Tracing the Call Stack

factorial(4) step by step:

```
factorial(4)
  → 4 * factorial(3)
    → 3 * factorial(2)
      → 2 * factorial(1)
        → 1 (base case!)
          ← 2 * 1 = 2
            ← 3 * 2 = 6
              ← 4 * 6 = 24
                = 24
```

Each call to `factorial` creates a new **stack frame** in memory. The calls pile up until the base case is reached, then they **unwind** — each returning its value back up the chain.



3. Tracing the Call Stack

Python's default recursion limit is 1000 frames. For very deep recursion, use iteration instead.



Outline

1. What is Recursion?
2. The Factorial Example
3. Tracing the Call Stack
- 4. Fibonacci Sequence**
5. Sum of a List Recursively
6. When to Use Recursion
7. Recursion vs Iteration Comparison



4. Fibonacci Sequence

Fibonacci: $F(n) = F(n-1) + F(n-2)$, with $F(0)=0$, $F(1)=1$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

```
def fibonacci(n):
    """Return the nth Fibonacci number (0-indexed)."""
    if n <= 0:
        return 0                # base case 1
    if n == 1:
        return 1                # base case 2
    return fibonacci(n - 1) + fibonacci(n - 2) # recursive case

for i in range(10):
    print(f"F({i}) = {fibonacci(i)}")
```



4. Fibonacci Sequence

Fibonacci: $F(n) = F(n-1) + F(n-2)$, with $F(0)=0$, $F(1)=1$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

```
def fibonacci(n):  
    """Return the nth Fibonacci number (0-indexed)."""  
    if n <= 0:  
        return 0                # base case 1  
    if n == 1:  
        return 1                # base case 2  
    return fibonacci(n - 1) + fibonacci(n - 2) # recursive case  
  
for i in range(10):  
    print(f"F({i}) = {fibonacci(i)}")
```

Warning: naive recursive Fibonacci is slow



4. Fibonacci Sequence

`fibonacci(40)` makes millions of calls. Use memoisation or iteration for large values.



Outline

1. What is Recursion?
2. The Factorial Example
3. Tracing the Call Stack
4. Fibonacci Sequence
- 5. Sum of a List Recursively**
6. When to Use Recursion
7. Recursion vs Iteration Comparison



5. Sum of a List Recursively

```
def recursive_sum(numbers):  
    """Return the sum of all numbers in the list."""  
    if len(numbers) == 0:          # base case – empty list  
        return 0  
    return numbers[0] + recursive_sum(numbers[1:]) # first + rest  
  
print(recursive_sum([1, 2, 3, 4, 5])) # 15  
print(recursive_sum([]))             # 0
```



5. Sum of a List Recursively



5. Sum of a List Recursively

```
def recursive_sum(numbers):  
    """Return the sum of all numbers in the list."""  
    if len(numbers) == 0:          # base case – empty list  
        return 0  
    return numbers[0] + recursive_sum(numbers[1:]) # first + rest  
  
print(recursive_sum([1, 2, 3, 4, 5])) # 15  
print(recursive_sum([]))             # 0
```

Tracing recursive_sum([1, 2, 3]):

```
recursive_sum([1, 2, 3])  
→ 1 + recursive_sum([2, 3])  
    → 2 + recursive_sum([3])  
        → 3 + recursive_sum([])  
            → 0
```



5. Sum of a List Recursively

$$\begin{aligned} & \leftarrow 3 + 0 = 3 \\ & \leftarrow 2 + 3 = 5 \\ & \leftarrow 1 + 5 = 6 \\ & = 6 \end{aligned}$$



Outline

1. What is Recursion?
2. The Factorial Example
3. Tracing the Call Stack
4. Fibonacci Sequence
5. Sum of a List Recursively
- 6. When to Use Recursion**
7. Recursion vs Iteration Comparison



6. When to Use Recursion

Good candidates for recursion:

- Problems naturally defined recursively (trees, nested structures)
- Divide-and-conquer algorithms (merge sort, binary search)
- Directory/file traversal
- Mathematical sequences with recursive definitions



6. When to Use Recursion



6. When to Use Recursion

Good candidates for recursion:

- Problems naturally defined recursively (trees, nested structures)
- Divide-and-conquer algorithms (merge sort, binary search)
- Directory/file traversal
- Mathematical sequences with recursive definitions

Usually better as iteration:

- Simple counting loops
- Accumulation (sum, product)
- Linear sequences where depth could be large



6. When to Use Recursion



6. When to Use Recursion

Good candidates for recursion:

- Problems naturally defined recursively (trees, nested structures)
- Divide-and-conquer algorithms (merge sort, binary search)
- Directory/file traversal
- Mathematical sequences with recursive definitions

Usually better as iteration:

- Simple counting loops
- Accumulation (sum, product)
- Linear sequences where depth could be large

Power function – elegant recursively

```
def power(base, exp):  
    if exp == 0:  
        return 1
```



6. When to Use Recursion

```
if exp % 2 == 0:
    half = power(base, exp // 2)
    return half * half          # efficient: avoids re-computing
return base * power(base, exp - 1)

print(power(2, 10))    # 1024
```



Outline

1. What is Recursion?
2. The Factorial Example
3. Tracing the Call Stack
4. Fibonacci Sequence
5. Sum of a List Recursively
6. When to Use Recursion
- 7. Recursion vs Iteration Comparison**



7. Recursion vs Iteration Comparison

Iterative sum

```
def sum_iter(n):  
    total = 0  
    for i in range(1, n + 1):  
        total += i  
    return total
```

Recursive sum

```
def sum_rec(n):  
    if n <= 0:  
        return 0  
    return n + sum_rec(n - 1)
```

Both give the same result

```
print(sum_iter(100))    # 5050  
print(sum_rec(100))     # 5050
```



7. Recursion vs Iteration Comparison

```
# Mathematical shortcut – also correct  
print(100 * 101 // 2) # 5050
```

For this particular problem, the iterative version is preferred — it handles large n without hitting the recursion limit.



Thanks for Watching - Any questions?

